

AI Testing Platform — Dashboard Guide

How a requirement becomes a running, committed Playwright test suite

The Big Picture

This is an **AI-powered test generation pipeline**. You write a requirement in plain English, and a chain of AI agents turns it into real, executable Playwright tests — committed to Git and runnable in CI/CD.

Requirement → Scenarios → Planner (live exploration) → Generator (writes Playwright code) →
Committed to Git → Run & Report

1. Requirements

You type a requirement in plain English (e.g. "A user should be able to log in with valid credentials"). This is stored in the database and triggers the **Intelligence Agent**, which analyzes it and proposes test scenarios.

2. Scenarios

The Intelligence Agent breaks your requirement into individual testable scenarios (e.g. "successful login", "invalid password", "empty fields"). Each scenario needs your approval — **Approve / Reject / Regenerate** — before moving forward. This human checkpoint stops the AI from running ahead on bad assumptions.

3. Playwright Planner — the exploration step

Once scenarios are approved, the **Planner Agent** launches a real headless Chrome browser and actually visits your configured app URL to explore it — discovering real routes, UI flows, and `data-testid` locators. This is the "Routes discovered / Flows discovered / Testids" panel on a requirement page. It grounds the scenarios in the real app instead of letting the model guess.

4. Generated Tests

The **Generator Agent** takes the grounded scenarios plus the discovered locators and writes a real `.spec.ts` Playwright file. Before it's accepted, it passes three checks:

- Syntax validation (TypeScript compiler)
- No hallucinated locators (only confirmed `data-testid` s are allowed)
- No duplicate test titles (Playwright silently refuses to run those)

Where does the generated code actually go?

There are **two separate repositories** on disk:

```
ai_testing/
├─ ai_testing/          ← the PLATFORM itself (dashboard, server, DB)
└─ generated-tests-repo/ ← a SEPARATE git repo holding ONLY your generated tests
```

When a test file is approved and committed:

1. It's written to disk at `generated-tests-repo/tests/generated/<slug>.spec.ts`
2. A real `git commit` is made in that separate repo — visible in the **"Git"** tab (e.g. `6a6fb98410` Add generated tests for: `...spec.ts`)
3. The platform's own database tracks metadata: version, status (`committed`), which agent run generated it

`generated-tests-repo` is a real, standalone Playwright project — its own `package.json` , `playwright.config.js` , and `node_modules` . It's meant to be pushed to GitHub/GitLab and wired into your own CI/CD independently of this dashboard.

How do you use the output?

Option A — From the dashboard

Click **"Run Test"** on any requirement page. This runs the real browser test against `generated-tests-repo` and reports pass/fail + duration into the **"CI/CD: Run & Report"** section.

Option B — Standalone, outside the dashboard

```
cd generated-tests-repo
npm run test           # runs ALL generated tests
npx playwright show-report # view the HTML report
```

Since it's a real git repo with real Playwright tests, you can push it to your own GitHub repo, wire it into GitHub Actions/Jenkins/etc., or hand it to a QA team — it has no dependency on this dashboard to run.

Option C — Auto-run after commit

Every commit automatically triggers a re-run ("auto (after commit)" entries in the CI/CD log), so you immediately know if a freshly generated test is broken.

The other tabs

Tab	What it's for
Agent Activity	Live log of every agent run (Intelligence, Planner, Generator) — completed/failed status, for debugging generation failures.
Git	Full commit history of <code>generated-tests-repo</code> — every commit the platform made, who triggered it, and PR status.
Settings	LLM provider, approval mode (manual/semi/full-auto), and the "Test App URL" field — tells the Planner/Generator/Runner which real application to target.

Key takeaway: the dashboard is the control panel that *produces* the tests, but the tests themselves live and run as an ordinary, independent Playwright project in `generated-tests-repo` — committed to git, ready to plug into any CI pipeline.